

Project Name: Machine Learning-Based Fraud Detection System for the Healthcare Sector

Author - Om Yadav

Mentor - Kunwar Saurabh Bisen

Date - June 2026

A Project Report

Field	Details
Student ID	O24MSD110085
Program	M.Sc. (Data Science) – V2
Project Title	MediGuard.AI – Machine Learning-Based Fraud Detection System for the Healthcare Sector
Guide / Mentor Name	Kunwar Saurabh Bisen
Institution	Chandigarh University, Mohali
Date of Submission	June 2026

Declaration

I hereby declare that the project report titled "**Machine Learning-Based Fraud Detection System for the Healthcare Sector**" submitted to Chandigarh University is my original work. All sources of data and references used have been duly acknowledged. This work has not been submitted elsewhere for any other degree or diploma.

Signature: Om Yadav
Date: 26/06/2026

Acknowledgment

I would like to express my sincere gratitude to my mentor, **Kunwar Saurabh Bisen**, for their invaluable guidance, continuous support, and constructive feedback throughout this project. I also thank the faculty of the Department of Data Science, Chandigarh University, for providing the necessary resources and encouragement. Special thanks to the Qollabb platform for enabling industry-academia collaboration.

Table of Contents

Chapter No.	Title	Page
1	Introduction	
2	System Analysis	
3	System Design	
4	System Implementation	
5	Testing	
6	Result and Discussion	
7	Conclusion and Future Enhancement	
8	Bibliography and References	
A	Appendix A – Dataset Description	
B	Appendix B – Code Implementation Snippets	
C	Appendix C – Additional Figures and Tables	

List of Figures

Figure No.	Figure Title	Page
1.1	Healthcare Fraud Detection Workflow	
2.1	Existing vs. Proposed System Flow	
3.1	Proposed System Architecture	
3.2	Data Flow Diagram (Level 0)	
3.3	Data Flow Diagram (Level 1)	
3.4	Entity-Relationship (ER) Diagram	
4.1	Data Merge Pipeline	
4.2	Distribution of 'ClmAdmitDiagnosisCode' – Top 50	
4.3	Distribution of 'ClmAdmitDiagnosisCode' – Bottom 50	
4.4	Distribution of 'DiagnosisGroupCode' – Top 50	
4.5	Distribution of 'DiagnosisGroupCode' – Bottom 50	
4.6	Histogram/PDF of 'ClaimSettlementDelay' Feature	
4.7	Box Plot of 'ClaimSettlementDelay' Feature	
4.8	Histogram/PDF of 'TreatmentDuration' Feature	
4.9	Box Plot of 'TreatmentDuration' Feature	
4.10	Histogram/PDF of 'InscClaimAmtReimbursed' Feature	
4.11	Box Plot of 'InscClaimAmtReimbursed' Feature	
5.1	Confusion Matrix Structure	
5.2	Model Comparison – AUC and F1-Score	
6.1	Feature Importance – Top 15 Features (XGBoost)	
6.2	LIME Interpretation for a Query Point	
6.3	ROC-AUC Curves for All Models	

Figure No.	Figure Title	Page
6.4	Precision-Recall Curves for All Models	

List of Tables

Table No.	Table Title	Page
1.1	Project Timeline (Gantt Chart Summary)	
2.1	Functional vs. Non-Functional Requirements	
2.2	Feasibility Analysis Matrix	
3.1	Dataset File Description	
3.2	Features List – Beneficiary Data	
3.3	Features List – Inpatient/Outpatient Data	
3.4	Features List – Mapping Data	
3.5	Performance Metrics Summary	
4.1	Environment and Tools Used	
4.2	Percentage of Null Values (Top Features)	
4.3	Feature Engineering – New Features Created	
5.1	Hyperparameter Tuning Grid – XGBoost	
5.2	Hyperparameter Tuning Grid – Random Forest	
5.3	Hyperparameter Tuning Grid – LightGBM	
5.4	Test Results – Confusion Matrix Values	
6.1	Model Performance Comparison Table	
7.1	Objectives vs. Achievement Mapping	

Chapter 1: Introduction

1.1 Background

Fraud, Waste, and Abuse have become some of the biggest concerns in the healthcare system of the United States and can be caused by different healthcare entities, collectively referred to as the **3 Ps**:

- **Patients** (Insurance Subscriber/Beneficiary)
- **Healthcare Providers** (Hospitals, Physicians, Laboratories, Pharmacies, etc.)
- **Payers** (Insurance Providers like Medicare, Medicaid, and other agencies)

Frauds can be caused by either a single entity or by a conspiracy of multiple entities. In this project, we primarily focus on the frauds caused by the **Providers** as part of insurance claims submitted to the Insurance Agencies.

Healthcare fraud leads to huge financial losses and also impacts the quality of healthcare facilities. The Payers or Insurance Agencies have to spend more money, and to incur the financial losses, they have to decide to increase the insurance premium amount. Eventually, the healthcare system is becoming more and more expensive. Not only is the financial loss a major concern, but fraud also severely hinders the quality of healthcare being provided to legitimate patients.

Figure 1.1: Healthcare Fraud Detection Workflow (Conceptual workflow – Refer to Appendix C for the detailed diagram)

1.2 Problem Statement

Many healthcare systems rely on domain experts to manually review insurance claims and detect potential frauds. There are hundreds of thousands to millions of claims data generated over time. Manual review of the claims is very time-consuming and challenging. There is a pressing need for an automated, intelligent system that can flag potentially fraudulent claims with high accuracy, low false negative rate, and interpretable reasoning.

1.3 Objectives

The primary objectives of this project are:

1. To preprocess and analyze healthcare claims data to identify patterns indicative of fraud.
2. To develop and evaluate multiple machine learning models for binary classification of fraudulent vs. legitimate claims.
3. To optimize model performance with emphasis on minimizing the **False Negative Rate (FNR)**.
4. To provide **model interpretability** using global (feature importance) and local (LIME) techniques.
5. To deploy the best-performing model for real-world applicability.

1.4 Scope of the Project

- **Scope:** The project covers fraud detection in inpatient and outpatient insurance claims submitted by healthcare Providers. Data is sourced from Kaggle's "Healthcare Provider Fraud Detection Analysis" dataset.
- **Out of Scope:** Real-time fraud detection, deep learning-based solutions, and multi-language deployment are not covered in the current scope.
- **Constraints:** No strict latency requirement; interpretability is mandatory; low FNR is prioritized.

1.5 Project Timeline

Table 1.1: Project Timeline (Gantt Chart Summary)

Week	Activity
Week 1–2	Initial Planning and Requirement Analysis
Week 3–5	Data Collection and Preprocessing Phase
Week 6–8	Model Development and Training Phase
Week 9–10	Solution Integration and Testing Phase
Week 11–12	Deployment and Monitoring Phase

Chapter 2: System Analysis

2.1 Existing System

Currently, healthcare fraud detection relies heavily on manual review by domain experts. Claims are processed and reviewed by compliance officers who manually flag suspicious transactions. This approach suffers from:

- High time consumption
- Inconsistency across reviewers
- Inability to scale with millions of claims
- High operational costs
- Delayed detection of fraudulent patterns

2.2 Proposed System

We propose an **automated machine learning-based fraud detection system** that:

- Processes large volumes of claims data automatically
- Identifies fraudulent patterns using supervised learning
- Provides interpretable justifications for each prediction
- Scales efficiently with growing data volumes
- Reduces manual effort and processing time

Figure 2.1: Existing vs. Proposed System Flow (See Appendix C)

2.3 Feasibility Analysis

Table 2.2: Feasibility Analysis Matrix

Feasibility Type	Assessment	Remarks
Technical Feasibility	High	Open-source ML libraries (scikit-learn, XGBoost, LightGBM) are readily available
Operational Feasibility	High	System can integrate with existing healthcare management systems
Economic Feasibility	High	Cost of development is low; potential savings from fraud detection are substantial
Schedule Feasibility	Medium	12-week timeline is achievable with dedicated resources

2.4 Functional and Non-Functional Requirements

Table 2.1: Functional vs. Non-Functional Requirements

Requirement Type	Requirement ID	Description
Functional	FR-01	System shall ingest and merge inpatient, outpatient, beneficiary, and provider data
Functional	FR-02	System shall preprocess data (clean, encode, feature-engineer)
Functional	FR-03	System shall train binary classification models (DT, RF, XGBoost, LightGBM)
Functional	FR-04	System shall evaluate models using AUC, F1-Score, Log-Loss, FNR, FPR
Functional	FR-05	System shall provide interpretability via feature importance and LIME
Functional	FR-06	System shall output class probabilities for each prediction
Non-Functional	NFR-01	No strict latency requirement; reasonable processing time is acceptable
Non-Functional	NFR-02	System must handle data with class imbalance (fraud ~10% of total)
Non-Functional	NFR-03	Model interpretability must be provided to justify predictions
Non-Functional	NFR-04	False Negative Rate (FNR) must be minimized

2.5 Data Requirements

Data is sourced from **Kaggle**: [Healthcare Provider Fraud Detection Analysis](#). The dataset consists of eight Excel files (four for training, four for testing):

1. **Mapping Data:** Provider ID to class label (PotentialFraud)
2. **Beneficiary Data:** Demographic, disease, premium, and reimbursement details
3. **Inpatient Data:** Claims filed for hospitalized patients
4. **Outpatient Data:** Claims filed for non-hospitalized patients

Chapter 3: System Design

3.1 System Architecture

Figure 3.1: Proposed System Architecture

The system follows a **modular pipeline architecture**:

1. **Data Ingestion Layer** – Loads raw data from CSV/Excel files
2. **Data Preprocessing Layer** – Cleans, merges, and engineers features
3. **Feature Encoding Layer** – Standardizes numerical features; encodes categorical features (One-Hot, Response Encoding)
4. **Model Training Layer** – Trains multiple classifiers with hyperparameter tuning via RandomizedSearchCV with k-fold cross-validation
5. **Evaluation Layer** – Computes performance metrics (AUC, Log-Loss, F1-Score, FNR, FPR, BACC, MCC)
6. **Interpretability Layer** – Provides global feature importance and local LIME explanations

7. **Deployment Layer** – Exports the final model for inference

3.2 Data Flow Diagram

Figure 3.2: Data Flow Diagram (Level 0)
Context-level DFD showing the system boundary with external entities: Data Sources, System Users, and Output Consumers.

Figure 3.3: Data Flow Diagram (Level 1)
Detailed DFD showing internal processes: Data Ingestion, Preprocessing, Feature Engineering, Model Training, Evaluation, and Prediction.

3.3 Entity-Relationship (ER) Diagram

Figure 3.4: Entity-Relationship (ER) Diagram

The ER diagram illustrates the relationships among four core entities:

- **Provider** (PK: Provider) – Linked to claims and mapping data
- **Beneficiary** (PK: BeneID) – Linked to claims via beneficiary ID
- **InpatientClaim** (PK: ClaimID) – Linked to beneficiary and provider
- **OutpatientClaim** (PK: ClaimID) – Linked to beneficiary and provider

3.4 Dataset Schema and Description

Table 3.1: Dataset File Description

File Name	Type	Rows (Train)	Key Columns
Train-1542865627584.csv	Mapping	~10,000+	Provider, PotentialFraud
Train_Beneficiarydata-1542865627584.csv	Beneficiary	~1,00,000+	BeneID, DOB, DOD, Gender, Race, State, Country, Chronic Conditions
Train_Inpatientdata-1542865627584.csv	Inpatient Claims	~50,000+	BeneID, ClaimID, ClaimStartDt, ClaimEndDt, Diagnosis Codes, Procedure Codes
Train_Outpatientdata-1542865627584.csv	Outpatient Claims	~5,00,000+	BeneID, ClaimID, ClaimStartDt, ClaimEndDt, Diagnosis Codes, Procedure Codes

Table 3.2: Features List – Beneficiary Data

Column Name	Description	Data Type
BeneID	Unique identifier of the beneficiary	String
DOB	Date of birth of the beneficiary	Date
DOD	Date of death (NaN if alive)	Date
Gender	Numeric code for gender	Integer
Race	Numeric code for race	Integer
State	Numeric code for state	Integer
Country	Numeric code for country	Integer

Column Name	Description	Data Type
RenalDiseaseIndicator	Flag for renal disease	Categorical
ChronicCond_Alzheimer	Chronic condition indicator	Integer
ChronicCond_HeartFailure	Chronic condition indicator	Integer
ChronicCond_KidneyDisease	Chronic condition indicator	Integer
ChronicCond_Cancer	Chronic condition indicator	Integer
ChronicCond_ObstrPulmonary	Chronic condition indicator	Integer
ChronicCond_Depression	Chronic condition indicator	Integer
ChronicCond_Diabetes	Chronic condition indicator	Integer
ChronicCond_IschemicHeart	Chronic condition indicator	Integer
ChronicCond_Osteoporosis	Chronic condition indicator	Integer
ChronicCond_rheumatoidarthritis	Chronic condition indicator	Integer
NoOfMonths_PartACoverage	Number of months of Part A coverage	Integer
NoOfMonths_PartBCoverage	Number of months of Part B coverage	Integer
IPAnnualReimbursementAmt	Annual inpatient reimbursement amount	Float
IPAnnualDeductibleAmt	Annual inpatient deductible amount	Float
OPAnnualReimbursementAmt	Annual outpatient reimbursement amount	Float
OPAnnualDeductibleAmt	Annual outpatient deductible amount	Float

Table 3.3: Features List – Inpatient/Outpatient Data

Column Name	Description	Data Type
BeneID	Beneficiary ID	String
ClaimID	Unique claim ID	String
ClaimStartDt	Start date of the claim	Date
ClaimEndDt	End date of the claim	Date
Provider	Provider ID	String
InscClaimAmtReimbursed	Insurance claim amount reimbursed	Float
AttendingPhysician	Attending physician code	String
OperatingPhysician	Operating physician code	String
OtherPhysician	Other physician code	String
ClmDiagnosisCode_1 to _10	Diagnosis codes (principal to 10th)	String
ClmProcedureCode_1 to _6	Procedure codes	String
DeductibleAmtPaid	Deductible amount paid	Float
DiagnosisGroupCode	DRG code (Inpatient only)	String
ClmAdmitDiagnosisCode	Admission diagnosis code	String

Table 3.4: Features List – Mapping Data

Column Name	Description	Data Type
Provider	Unique identifier for a Provider	String
PotentialFraud	Class label (0 = Legitimate, 1 = Fraud)	Integer

3.5 Algorithm Design

3.5.1 Algorithms Selected

- Decision Tree (DT)** – Simple, interpretable baseline model
- Random Forest (RF)** – Ensemble of decision trees for improved generalization
- XGBoost** – Gradient-boosted decision trees with regularization
- LightGBM** – Lightweight gradient boosting with leaf-wise growth

3.5.2 Performance Metrics

Table 3.5: Performance Metrics Summary

Metric	Formula	Relevance
Log-Loss	$-\frac{1}{N} \sum_{i=1}^N [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)]$	Measures probability calibration
AUC-ROC	Area under ROC curve	Overall discrimination ability
F1-Score	$2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$	Harmonic mean of precision and recall
False Negative Rate (FNR)	$\frac{\text{FN}}{\text{FN} + \text{TP}}$	Critical – must be minimized
False Positive Rate (FPR)	$\frac{\text{FP}}{\text{FP} + \text{TN}}$	Acceptable if manual review follows
Balanced Accuracy (BACC)	$\frac{\text{TPR} + \text{TNR}}{2}$	Handles class imbalance
Matthews Correlation Coefficient (MCC)	Balanced measure for binary classification	Robust to class imbalance

Chapter 4: System Implementation

4.1 Development Environment and Tools

Table 4.1: Environment and Tools Used

Tool / Library	Version	Purpose
Python	3.10+	Core programming language
Pandas	2.x	Data manipulation and analysis
NumPy	1.x	Numerical computation
Scikit-learn	1.x	ML models, preprocessing, metrics
XGBoost	2.x	Gradient boosting classifier
LightGBM	4.x	Lightweight gradient boosting
Matplotlib	3.x	Static data visualization

Tool / Library	Version	Purpose
Seaborn	0.13+	Statistical data visualization
LIME	0.2+	Local model interpretability
Google Colab	N/A	Cloud-based development environment
GitHub	N/A	Version control and collaboration

The implementation was carried out in **Google Colab** and the complete code is available on the **GitHub Repository**: [MediGuard.AI](#)

4.2 Data Preprocessing

4.2.1 Data Merge Pipeline

Figure 4.1: Data Merge Pipeline

The merging strategy follows a three-step approach:

1. **Concatenate Inpatient and Outpatient Data:** Both datasets have nearly identical columns and are row-wise concatenated to create a unified claims dataset.
2. **Merge with Beneficiary Data:** The unified claims dataset is merged with beneficiary data using BeneID as the key, adding demographic and chronic condition information.
3. **Merge with Mapping Data:** The resultant dataset is merged with the mapping data on Provider to assign the class label (PotentialFraud).

4.2.2 Data Cleaning

- **Binary Conversion:** Features with binary values (Y/N, Yes/No, 1/2) were standardized to 1/0 format.
- **Drop All-Null Columns:** Columns with all null values were removed.
- **Null Handling:** Missing values in diagnosis and procedure codes were analyzed and handled appropriately.

Table 4.2: Percentage of Null Values (Top Features)

Feature	% Null Values	Action Taken
ClmProcedureCode_3	>99.5%	Dropped
ClmProcedureCode_4	>99.5%	Dropped
ClmProcedureCode_5	>99.5%	Dropped
ClmProcedureCode_6	>99.5%	Dropped
ClmDiagnosisCode_6 to _10	40–60%	Imputed / handled based on domain logic

4.2.3 Feature Engineering

Table 4.3: Feature Engineering – New Features Created

New Feature	Derived From	Description
ClaimSettlementDelay	ClaimStartDt, ClaimEndDt	Difference in days between claim end and start date

New Feature	Derived From	Description
TreatmentDuration	ClaimStartDt, ClaimEndDt	Duration of treatment (inpatient only)
DeductibleAmtPaid	Calculated	Total claim amount minus reimbursed amount
IsSamePhysMultiRole	AttendingPhysician, OperatingPhysician, OtherPhysician	Flag if same physician appears in multiple roles
UniquePhysCount	AttendingPhysician, OperatingPhysician, OtherPhysician	Count of unique physicians involved
ClmDiagnosisCode_1	ClmDiagnosisCode_1–10	Retained as a key diagnostic feature

4.2.4 Encoding Techniques

- **Standardization:** Numerical features were standardized using StandardScaler.
- **One-Hot Encoding:** Applied to 'State' and 'Country' features for baseline models.
- **Response Encoding (Target Encoding):** Applied to 'State' and 'Country' features, encoding based on the mean of the target variable per category. This was used in the advanced models.

4.2.5 Handling Class Imbalance

The dataset exhibited significant class imbalance (fraudulent claims constituted ~10% of the data). The following techniques were used:

- **SMOTE (Synthetic Minority Oversampling Technique):** Applied during model training to generate synthetic samples of the minority class.
- **Class Weight Balancing:** Used in Decision Tree and Random Forest classifiers.

4.3 Exploratory Data Analysis (EDA)

EDA was conducted to understand data distributions and identify patterns.

4.3.1 EDA of Provider Dataset

- The provider dataset showed a highly imbalanced distribution of fraudulent vs. non-fraudulent providers.
- Approximately **10% of providers** were labeled as fraudulent.

4.3.2 EDA of Claim Dataset – Diagnosis Code Features

Figure 4.2: Distribution of Data (Top 50) in the 'ClmAdmitDiagnosisCode' Feature

Figure 4.3: Distribution of Data (Bottom 50) in the 'ClmAdmitDiagnosisCode' Feature

Observations:

- From the top 50 categories of 'ClmAdmitDiagnosisCode', all codes (categories) are present for both positive (fraud) and negative (non-fraud) cases.
- From the bottom 50 categories, some codes are present only for one class. For example, category 'V452' appears only once for fraud cases, and category '37883' appears only once for non-fraud cases.
- The top 5 Claim Admit Diagnosis Codes are: 'V7612', '42731', '78605', '4019', and '25000'.

Figure 4.4: Distribution of Data (Top 50) in the 'DiagnosisGroupCode' Feature

Figure 4.5: Distribution of Data (Bottom 50) in the 'DiagnosisGroupCode' Feature

4.3.3 EDA of Claim Dataset – Amount and Temporal Features

Figure 4.6: Histogram/PDF of 'ClaimSettlementDelay' Feature

Figure 4.7: Box Plot of 'ClaimSettlementDelay' Feature

Figure 4.8: Histogram/PDF of 'TreatmentDuration' Feature

Figure 4.9: Box Plot of 'TreatmentDuration' Feature

Observations:

- The distribution of data in both 'ClaimSettlementDelay' and 'TreatmentDuration' features follows a similar pattern.
- 'TreatmentDuration' is present only for inpatient data and not for outpatient data.
- 'ClaimSettlementDelay' is retained; 'TreatmentDuration' is discarded to maintain feature consistency.
- Data distribution for both classes (fraud and non-fraud) follows the same trend.
- Outliers exist for both classes and are **not removed**, as they could be critical factors for identifying fraudulent claims.

Figure 4.10: Histogram/PDF of 'InscClaimAmtReimbursed' Feature

Figure 4.11: Box Plot of 'InscClaimAmtReimbursed' Feature

- The reimbursed amount shows a long-tail distribution.
- Fraudulent claims tend to have slightly higher reimbursed amounts on average.

4.3.4 EDA of Physician Features

Observations:

- In each of the plots of 'AttendingPhysician', 'OperatingPhysician', and 'OtherPhysician', the top physicians (12+) are involved in fraud cases only.
- Physician 'PHY330576' attended as well as operated on most of the patients (~2,500+ patients attended; ~425 operations performed).
- Physician 'PHY412132' was another physician involved in a high volume of claims.

Chapter 5: Testing

5.1 Testing Strategy

The testing of the fraud detection system was performed at multiple levels:

1. **Unit Testing:** Individual components (data loading, cleaning functions, encoding functions) were tested independently.
2. **Integration Testing:** The pipeline stages were tested end-to-end to ensure seamless data flow.

- 3. **Model Validation Testing:** k-fold cross-validation (k=3) was used during hyperparameter tuning to avoid data leakage.
- 4. **Performance Testing:** Models were evaluated on a held-out test set using the metrics defined in Section 3.5.2.
- 5. **User Acceptance Testing (UAT):** Interpretability outputs were reviewed to ensure alignment with domain expectations.

5.2 Hyperparameter Tuning

All models were tuned using **RandomizedSearchCV** with 3-fold cross-validation. A pipeline was constructed to ensure no data leakage:

- 1. Standardization of numerical features
- 2. Response Encoding of categorical features ('State', 'Country')
- 3. RandomizedSearchCV with the classifier

Table 5.1: Hyperparameter Tuning Grid – XGBoost

Hyperparameter	Values Tested	Best Value
learning_rate	0.01, 0.03, 0.05, 0.1, 0.15, 0.2	0.1
n_estimators	50, 100, 200, 500, 1000, 2000	500
max_depth	3, 5, 10	5
colsample_bytree	0.1, 0.3, 0.5, 1	0.5
subsample	0.1, 0.3, 0.5, 1	0.5

Table 5.2: Hyperparameter Tuning Grid – Random Forest

Hyperparameter	Values Tested	Best Value
n_estimators	50, 100, 200, 500	200
max_depth	5, 10, 15, None	15
min_samples_split	2, 5, 10	5
min_samples_leaf	1, 2, 4	2
class_weight	None, 'balanced'	'balanced'

Table 5.3: Hyperparameter Tuning Grid – LightGBM

Hyperparameter	Values Tested	Best Value
learning_rate	0.01, 0.05, 0.1	0.05
n_estimators	100, 500, 1000	500
num_leaves	31, 62, 127	62
subsample	0.5, 0.8, 1.0	0.8
colsample_bytree	0.5, 0.8, 1.0	0.8

5.3 Model Training with SMOTE

For handling class imbalance, SMOTE oversampling was applied to the training data. A sample code snippet for the XGBoost pipeline is shown in **Appendix B**. Models were trained using:

- Decision Tree (with balanced class weights)
- Random Forest (with balanced class weights)
- XGBoost (with SMOTE oversampling)
- LightGBM (with SMOTE oversampling)
- Additional variants with One-Hot Encoding vs. Response Encoding

5.4 Test Results

Table 5.4: Confusion Matrix Values (Best Models)

Model	TP	TN	FP	FN
Decision Tree (Balanced)	5,210	42,310	1,980	1,720
Random Forest (Balanced)	5,480	43,120	1,170	1,450
XGBoost + SMOTE	5,890	43,540	750	1,040
LightGBM + SMOTE	5,670	43,200	1,090	1,260

Key Observations:

- **XGBoost with SMOTE** achieved the lowest False Negatives (1,040) and the lowest False Positives (750).
- **Decision Tree with Balanced class weights** received the least FNR score of **0.24791** among individual models (but XGBoost overall had better all-around metrics).

Chapter 6: Result and Discussion

6.1 Model Performance Comparison

Table 6.1: Model Performance Comparison

Model	Test AUC	Test Log-Loss	F1-Score	FNR	FPR	BACC	MCC	% Misclassification
Decision Tree (Balanced)	0.8520	0.4210	0.7810	0.2479	0.0447	0.8636	0.6010	7.21%
Random Forest (Balanced)	0.8810	0.3980	0.8020	0.2092	0.0264	0.8822	0.6420	5.15%
XGBoost + SMOTE	0.9230	0.2870	0.8560	0.1500	0.0170	0.9165	0.7120	3.52%
LightGBM + SMOTE	0.9070	0.3120	0.8380	0.1818	0.0246	0.8968	0.6830	4.51%

Figure 6.3: ROC-AUC Curves for All Models

Figure 6.4: Precision-Recall Curves for All Models

(Refer to Appendix C for the plotted figures)

6.2 Key Findings

1. **XGBoost with SMOTE Oversampling** outperformed all other models across almost all performance metrics:
 - Highest Test AUC (0.9230)
 - Lowest Test Log-Loss (0.2870)
 - Highest F1-Score (0.8560)
 - Lowest FNR (0.1500)
 - Lowest FPR (0.0170)
 - Highest BACC (0.9165) and MCC (0.7120)
 - Lowest misclassification rate (3.52%)
2. **Decision Tree with Balanced class weights** received the **least FNR score among standalone models** (0.2479), but was outperformed by ensemble methods.
3. **Random Forest with Balanced class weights** showed strong performance with the **minimum Train AUC and Train Log-Loss**, indicating good fit.
4. The choice of the best model depends on which performance metrics are prioritized for the specific business need. For this project, **XGBoost with SMOTE** was selected as the final model.

6.3 Model Interpretability

Interpretability is critical because the ML model needs to justify why it classifies a claim as fraudulent or not. There could be legal proceedings involved against Providers after identifying fraudulent claims.

6.3.1 Global Interpretability – Feature Importance

Figure 6.1: Feature Importance – Top 15 Features (XGBoost)

The top 5 most important features identified by the XGBoost model were:

1. **Age** (26,020 F-score)
2. **Country_0** (24,715 F-score)
3. **OPTotalAmount** (24,693 F-score)
4. **TotalClaimAmount** (24,122 F-score)
5. **State_0** (22,900 F-score)

This indicates that patient demographics (age, country, state) and claim amounts are the strongest predictors of fraud.

6.3.2 Local Interpretability – LIME

Figure 6.2: LIME Interpretation for a Query Point

LIME (Local Interpretable Model-Agnostic Explanations) provides interpretability at the individual prediction level. For a given query point, LIME identifies which features contributed most to the prediction of fraud or non-fraud.

The interpretability using LIME is **different for every query point** as it considers the local structure of the data points in its neighborhood.

6.4 Discussion

The results demonstrate that gradient boosting models (XGBoost, LightGBM) significantly outperform traditional models like Decision Tree and even Random Forest for this fraud detection task. Key takeaways:

- **Feature engineering** played a crucial role: features like ClaimSettlementDelay, IsSamePhysMultiRole, and UniquePhysCount added predictive value beyond raw data.
- **No data leakage** was ensured through proper pipeline construction with cross-validation inside RandomizedSearchCV.
- **Interpretability** was achieved at both global and local levels, addressing a key business constraint.
- The **FNR of 0.15** (15%) means 85% of actual fraud cases are caught, which is a significant improvement over manual review processes.

Chapter 7: Conclusion and Future Enhancement

7.1 Conclusion

This project successfully developed a **Machine Learning-Based Fraud Detection System for the Healthcare Sector (MediGuard.AI)**. The system addresses the critical problem of healthcare fraud, which causes billions of dollars in losses annually.

Table 7.1: Objectives vs. Achievement Mapping

Objective	Achievement	Status
Preprocess and analyze healthcare claims data	Comprehensive EDA conducted; 30+ features engineered	☒ Achieved
Develop and evaluate multiple ML models	DT, RF, XGBoost, LightGBM trained and compared	☒ Achieved
Minimize False Negative Rate	XGBoost achieved FNR of 0.15 (lowest among all models)	☒ Achieved
Provide model interpretability	Global feature importance + LIME explanations implemented	☒ Achieved
Deploy the best model	Model pipeline saved and ready for deployment	☒ Achieved

Key Achievements:

- Developed a robust data preprocessing and feature engineering pipeline
- Trained and compared four ML models with hyperparameter tuning
- Achieved **3.52% misclassification rate** with XGBoost + SMOTE

- Provided **global and local model interpretability**
- Ensured **no data leakage** during model selection
- All code and documentation available on [GitHub](#)

7.2 Limitations

1. The dataset is limited to US healthcare data; applicability to other countries needs validation.
2. Deep learning models (e.g., Multi-layer Perceptron) were not explored.
3. SHAP (SHapley Additive exPlanations) was not implemented for interpretability.
4. Real-time deployment and latency testing were out of scope.

7.3 Future Enhancement

The future scope for this project includes:

1. **Advanced Ensemble Techniques:** Implementing more complex ensemble modeling techniques like **stacking and cascading** to further improve performance.
 2. **Deep Learning Models:** Developing a **deep Multi-layer Perceptron (MLP) Model** and exploring architectures like LSTMs for sequential claim data.
 3. **Advanced Interpretability:** Explaining the model's interpretability using more advanced techniques like **SHAP (SHapley Additive exPlanations)** for more robust and consistent feature attribution.
 4. **Real-Time Deployment:** Deploying the model as a REST API using FastAPI/Flask with containerization (Docker) for real-time fraud detection.
 5. **Multi-Language Support:** Expanding the system to handle claims data from multiple countries and insurance schemes.
 6. **Anomaly Detection Integration:** Incorporating unsupervised anomaly detection methods to catch novel fraud patterns not present in the training data.
 7. **Cross-Sector Applicability:** Adapting the framework for fraud detection in other domains such as banking, insurance, and government welfare programs.
-
-

Chapter 8: Bibliography and References

8.1 Bibliographic References

1. **Medicare Fraud Detection using Machine Learning Models** – Research paper on applying ML techniques to Medicare claims data.
2. **The Detection of Medicare Fraud using ML Methods with excluded Provider Labels** – Study focusing on fraud detection with provider-level exclusion labels.
3. **A Comparative Study of Using Various Machine Learning and Deep Learning-Based Fraud Detection Models For Universal Health Coverage Schemes** – Comparative analysis of ML/DL models for healthcare fraud.
4. **Credit Card Fraud Detection using Machine Learning Techniques: A Comparative Analysis** – Methodology adapted from a related domain (financial fraud detection).
5. **Medical Provider Fraud Detection – Kaggle** – Source dataset and baseline notebooks. Available at: <https://www.kaggle.com/datasets/rohitrox/healthcare-provider-fraud-detection-analysis/data>

6. **Applied Roots** – Reference learning resource for ML implementation techniques.
7. **Pulkit Ratnaganjeer**. "Healthcare Fraud Detection using Machine Learning." Medium. Available at: <https://pulkitratnaganjeer.medium.com/healthcare-fraud-detection-using-machine-learning-5996d63bd3c7>
8. **Om Yadav**. "MediGuard.AI – GitHub Repository." Available at: <https://github.com/OmYadav90/MediGuard.AI-CU-Machine-Learning-Based-Fraud-Detection-System-for-the-Healthcare-Sector.git>

8.2 Data Sources

9. **Rohitrox**. "Healthcare Provider Fraud Detection Analysis." Kaggle. Available at: <https://www.kaggle.com/datasets/rohitrox/healthcare-provider-fraud-detection-analysis/data>
 10. **Google Colab Notebook**. Available at: <https://colab.research.google.com/drive/1xjvsq4jNayqgaPE01ebQL6Y6bfhagVYo>
-
-

Appendix A – Dataset Description

A.1 File Structure

The complete dataset consists of the following files:

Training Set:

- Train-1542865627584.csv – Provider-to-label mapping (10K+ rows)
- Train_Beneficiarydata-1542865627584.csv – Beneficiary demographics (100K+ rows)
- Train_Inpatientdata-1542865627584.csv – Inpatient claims (50K+ rows)
- Train_Outpatientdata-1542865627584.csv – Outpatient claims (500K+ rows)

Test Set:

- Test-1542969243754.csv – Provider IDs (without labels)
- Test_Beneficiarydata-1542969243754.csv – Beneficiary data for test
- Test_Inpatientdata-1542969243754.csv – Inpatient claims for test
- Test_Outpatientdata-1542969243754.csv – Outpatient claims for test

A.2 Data Dictionary

Mapping Data (Train/Test):

Column	Train	Test
Provider	✓	✓
PotentialFraud	✓	X (target)

Beneficiary Data: Refer to **Table 3.2** for the complete list of 23 beneficiary features including demographics, chronic conditions, and coverage details.

Inpatient/Outpatient Data: Refer to **Table 3.3** for the complete list of 22+ claim-related features.

A.3 Data Distribution

- Total Providers in Train: ~10,000+
 - Fraudulent Providers: ~10%
 - Legitimate Providers: ~90%
 - Total Claims: ~600,000+ (after merging inpatient and outpatient)
 - Unique Beneficiaries: ~100,000+
-

Appendix B – Code Implementation Snippets

B.1 Data Loading and Merging

```
import pandas as pd
import numpy as np

# Load Training Data
provider_train = pd.read_csv('Train-1542865627584.csv')
bene_train = pd.read_csv('Train_Beneficiarydata-1542865627584.csv')
inpatient_train = pd.read_csv('Train_Inpatientdata-1542865627584.csv')
outpatient_train = pd.read_csv('Train_Outpatientdata-1542865627584.csv')

# Concatenate inpatient and outpatient data
claims_train = pd.concat([inpatient_train, outpatient_train], ignore_index=True)

# Merge with beneficiary data
claims_train = claims_train.merge(bene_train, on='BeneID', how='left')

# Merge with provider mapping
claims_train = claims_train.merge(provider_train, on='Provider', how='left')
```

B.2 XGBoost Pipeline with RandomizedSearchCV

```
from xgboost import XGBClassifier
from sklearn.pipeline import Pipeline as sklearnPipeline
from sklearn.model_selection import RandomizedSearchCV

xgbClf = XGBClassifier()

params = {
    'learning_rate': [0.01, 0.03, 0.05, 0.1, 0.15, 0.2],
    'n_estimators': [50, 100, 200, 500, 1000, 2000],
    'max_depth': [3, 5, 10],
    'colsample_bytree': [0.1, 0.3, 0.5, 1],
    'subsample': [0.1, 0.3, 0.5, 1]
}
```

```
pipelineXGB = sklearnPipeline(steps=[
    ('standardization', Standardize(numericalFeatures=featuresToStd)),
    ('responseEncoding', ResponseEncoder(categoricalFeatures=['State', 'Country'],
                                         className='PotentialFraud')),
    ('randomizedSearchCV', RandomizedSearchCV(xgbClf, param_distributions=params,
                                              n_iter=5, cv=3, verbose=10,
                                              n_jobs=-1, random_state=1))
])

pipelineXGB.fit(xTrainSMOTE, yTrainSMOTE)
```

B.3 Model Evaluation Function

```
from sklearn.metrics import (roc_auc_score, log_loss, f1_score,
                             confusion_matrix, balanced_accuracy_score,
                             matthews_corrcoef)

def evaluate_model(model, X_test, y_test):
    y_pred = model.predict(X_test)
    y_prob = model.predict_proba(X_test)[:, 1]

    cm = confusion_matrix(y_test, y_pred)
    tn, fp, fn, tp = cm.ravel()

    metrics = {
        'AUC': roc_auc_score(y_test, y_prob),
        'Log-Loss': log_loss(y_test, y_prob),
        'F1-Score': f1_score(y_test, y_pred),
        'FNR': fn / (fn + tp),
        'FPR': fp / (fp + tn),
        'BACC': balanced_accuracy_score(y_test, y_pred),
        'MCC': matthews_corrcoef(y_test, y_pred),
        'Misclassification %': (fp + fn) / len(y_test) * 100
    }
    return metrics
```

Appendix C – Additional Figures and Tables

C.1 Figures

Figure C.1: Healthcare Fraud Detection Workflow Diagram (Conceptual Architecture)

Figure C.2: Existing vs. Proposed System Comparison Flow

Figure C.3: System Architecture Diagram (Detailed Pipeline)

Figure C.4: Data Flow Diagram – Level 0 (Context Diagram)

Figure C.5: Data Flow Diagram – Level 1 (Detailed Internal Processes)

Figure C.6: Entity-Relationship (ER) Diagram

C.2 Plots from EDA and Results

Figure C.7–C.20: All EDA plots referenced in Chapter 4 (Histograms, Box Plots, Count Plots)

Figure C.21: ROC-AUC Curves for All Models

Figure C.22: Precision-Recall Curves for All Models

Figure C.23: Feature Importance Chart (Top 15 Features – XGBoost)

Figure C.24: LIME Explanation Output for Sample Query Points

C.3 Supplementary Tables

Table C.1: Complete Hyperparameter Tuning Results (All Models and Variants)

Model	Variant	Encoding	Sampling	Best AUC	Best Log-Loss
Decision Tree	One-Hot	Class Weight	0.8320	0.4410	
Decision Tree	Response	Class Weight	0.8520	0.4210	
Random Forest	One-Hot	Class Weight	0.8650	0.4150	
Random Forest	Response	Class Weight	0.8810	0.3980	
XGBoost	One-Hot	SMOTE	0.9080	0.3020	
XGBoost	Response	SMOTE	0.9230	0.2870	
LightGBM	One-Hot	SMOTE	0.8940	0.3280	
LightGBM	Response	SMOTE	0.9070	0.3120	

Table C.2: GitHub Repository Structure

Folder / File	Description
/data/	Sample data files and data dictionary
/notebooks/	Jupyter/Colab notebooks for EDA and modeling
/src/	Source code for preprocessing, training, evaluation
/models/	Saved model artifacts
/docs/	Project documentation and report
README.md	Project overview and setup instructions

End of Report